

Multimodal AI E-commerce Intelligence & Automation Agent

Project Proposal

AI/ML	SaaS Platform	6-Week MVP	Zero-Cost Local Setup
-------	---------------	------------	-----------------------

Technology Stack

Frontend	Next.js + React + Tailwind CSS
Backend	FastAPI + Python 3.11
Databases	PostgreSQL + Qdrant (vector)
VLM	Qwen2.5-VL 3B via Ollama
Agents	LangGraph + MCP Server
MLOps	MLflow + Prompt versioning

1. Executive Summary

This project proposes the development of a production-style AI SaaS platform for e-commerce sellers. The system automates product listing creation, product understanding, semantic search, recommendation, marketing content generation, and customer-support assistance.

The platform uses **Qwen2.5-VL 3B through Ollama** as the Vision-Language Model to analyze product images and extract structured e-commerce metadata — including title, category, color, material, brand, and description. The system also incorporates a Retrieval-Augmented Generation pipeline using **Qdrant** as the vector database, **PostgreSQL** as the main database, **FastAPI** as the backend, **Next.js** as the frontend, **LangGraph** for agent workflows, and an **MCP server** as the agent tool layer.

The entire project is designed to be achievable in **six weeks** using a zero-cost local development setup, while following production-grade architectural principles throughout.

2. Problem Statement

Small and medium e-commerce sellers spend significant time on manual, repetitive tasks: writing product descriptions, creating listings from images, generating ad copy, and answering the same customer questions. This creates the following pain points:

- Product listing creation is slow and inconsistent
- SEO titles and ad copy must be written manually for every product
- Customer support teams repeatedly answer the same catalog questions
- Product catalogs become difficult to search semantically as they grow
- Small sellers cannot afford enterprise automation systems
- General AI tools are useful but disconnected from the seller's database and workflow

3. Why This Product Over General AI Tools?

General AI chatbots are powerful but not built as complete e-commerce workflow systems. A seller using a chatbot still manually uploads images, writes prompts, copies results, organizes data, and manages the workflow themselves. This SaaS product turns AI into a **structured automation system**.

User writes prompts manually	Uses fixed e-commerce workflows
No built-in product database	Stores products in PostgreSQL
No catalog vector search by default	Qdrant semantic product search
No automatic listing pipeline	Image-to-listing workflow
No dedicated seller dashboard	Full e-commerce dashboard
No structured product schema	Extracts title, category, color, material

No similar product system	Automatic product recommendations
No built-in customer support RAG	Answers from seller's own catalog
No agent workflow logs	Stores agent decisions and tool calls

4. Project Goal

Build a working AI-powered e-commerce assistant that can:

- Understand product images using a real Vision-Language Model
- Extract structured product information automatically
- Create editable product listing drafts
- Store product data in PostgreSQL with full CRUD support
- Store product embeddings in Qdrant for semantic retrieval
- Enable semantic product search across the catalog
- Recommend similar products based on vector similarity
- Generate marketing content: descriptions, SEO titles, ad copy
- Answer customer questions using RAG over the product catalog
- Use LangGraph agents to automate multi-step workflows
- Log agent activity and model outputs for evaluation

5. System Architecture

Next.js Frontend	Product Upload · Dashboard · Search · Marketing · Support
FastAPI Backend	Product · Image · VLM · Search · Marketing · Support · Agent APIs
Data Layer	PostgreSQL (products, listings, agents) + Qdrant (vector embeddings)
Ollama Model Layer	Qwen2.5-VL 3B (vision) + Local LLM (text generation)
LangGraph Agents	Planner · Vision · Marketing · Support · Tool agents
MCP Tool Server	create/get/update product · search · save listing · mock messaging
MLOps / LLOps	MLflow · Prompt versioning · Agent decision logs · Evaluation metrics

6. Core Features

6.1 Product Understanding via VLM

A seller uploads a product image. **Qwen2.5-VL 3B** analyzes the image and returns structured JSON metadata. The seller can review and edit before saving. The VLM handles product recognition, visible text extraction, brand detection, color/material estimation, category prediction, and description

generation.

Sample VLM output:

```
{ "title": "Black Running Shoes", "category": "Footwear",  
  "color": "Black", "material": null, "brand": null,  
  "attributes": { "style": "sports", "type": "shoes" },  
  "description": "A pair of black sports running shoes.",  
  "confidence": 0.78 }
```

6.2 Product RAG System

Each saved product is converted to searchable text, embedded using **sentence-transformers/all-MiniLM-L6-v2**, and indexed in Qdrant. This enables semantic search, similar product retrieval, product recommendations, and grounded customer-support answers.

6.3 Marketing Content Generation

- Product descriptions
- SEO-optimized titles
- Instagram ad copy
- Facebook ad copy
- Short promotional text

6.4 Customer Support Agent (RAG)

The support agent answers customer questions by retrieving relevant products from Qdrant and generating a grounded answer via the local LLM. Hallucination is reduced by strictly using only retrieved product context. If retrieval confidence is low, the system returns an explicit 'not enough information' response.

6.5 Agentic Workflow System (LangGraph)

Agent	Responsibility
Planner Agent	Breaks user tasks into smaller executable steps
Vision Agent	Calls Qwen2.5-VL to extract product info from images
Marketing Agent	Generates descriptions, SEO titles, and ad copy
Customer Support Agent	Answers questions using RAG over the product catalog
Tool Agent	Calls tools through the MCP server layer

6.6 MCP Tool Server

The MCP server exposes discrete tools that agents call. It acts as the agent tool layer — FastAPI remains the SaaS backend while MCP handles all agent-facing operations:

- create_product / get_product / update_product
- search_products / similar_products
- save_listing / generate_marketing_content
- save_agent_log / send_mock_email / send_mock_whatsapp

7. Database Design

Main PostgreSQL tables:

- users, products, product_images, product_attributes
- listings, marketing_assets
- customer_conversations, customer_messages
- agent_runs, agent_steps
- prompt_versions, eval_runs, usage_events

Products table (key fields):

```
id UUID PRIMARY KEY
title TEXT NOT NULL
category TEXT, brand TEXT, color TEXT, material TEXT
description TEXT, price NUMERIC(12,2), currency TEXT
status TEXT DEFAULT 'draft'
qdrant_point_id TEXT
created_at / updated_at TIMESTAMP
```

8. API Design

Product APIs

- POST /api/v1/products
- GET /api/v1/products
- GET /api/v1/products/{id}
- PATCH /api/v1/products/{id}
- DELETE /api/v1/products/{id}

VLM API

- POST /api/v1/vlm/extract-product

Search APIs

- POST /api/v1/search/products
- POST /api/v1/search/similar
- POST /api/v1/search/recommendations

Marketing APIs

- POST /api/v1/marketing/generate-description
- POST /api/v1/marketing/generate-ad
- POST /api/v1/marketing/generate-seo-title

Support APIs

- POST /api/v1/support/answer
- POST/GET /api/v1/support/conversations

Agent APIs

- POST /api/v1/agents/run

- GET /api/v1/agents/runs
- GET /api/v1/agents/runs/{id}

9. Main Workflows

Workflow 1: Image to Product Listing

1. Seller uploads product image
2. FastAPI receives and saves image locally
3. Qwen2.5-VL analyzes image and extracts product JSON
4. Seller reviews and edits the product draft
5. Product saved in PostgreSQL
6. Product embedding generated and stored in Qdrant
7. Listing becomes semantically searchable

Workflow 2: Customer Query to RAG Answer

1. Customer submits a natural language question
2. Query is embedded with the same embedding model
3. Qdrant retrieves top-k relevant products
4. Support Agent reads retrieved context
5. Local LLM generates a grounded answer
6. Answer returned to the customer

Workflow 3: Product Recommendation

1. Seller selects a product in the dashboard
2. Product vector retrieved from Qdrant
3. Similar vectors searched using cosine similarity
4. Backend applies optional filters (category, price range)
5. Recommended products displayed in the frontend

Workflow 4: Marketing Content Generation

1. Seller selects a product
2. Product data fetched from PostgreSQL
3. Marketing Agent generates SEO title and ad copy
4. Result saved as a marketing asset
5. Seller can copy, edit, or export the content

10. Six-Week Implementation Plan

Week 1	Project Foundation
Tasks	• GitHub repo, FastAPI backend, Next.js frontend • PostgreSQL with Docker, product database models • Product CRUD APIs, basic dashboard layout
Deliverables	✓ Running backend & frontend ✓ PostgreSQL connected ✓ Product CRUD working
Week 2	VLM Product Understanding
Tasks	• Install Ollama, pull Qwen2.5-VL 3B • Build VLM service and image upload endpoint • Extract structured product JSON from images
Deliverables	✓ Image upload working ✓ Qwen2.5-VL integration ✓ Product extraction working
Week 3	Qdrant & Semantic Search
Tasks	• Add Qdrant via Docker, add embedding model • Convert product data to searchable text • Generate and store vectors, build search APIs
Deliverables	✓ Product indexing working ✓ Semantic search working ✓ Similar product recommendations
Week 4	Marketing & Support RAG
Tasks	• Add local text LLM via Ollama • Generate product descriptions, SEO titles, ad copy • Build customer support RAG flow using Qdrant retrieval
Deliverables	✓ Marketing generation working ✓ Customer support answers working
Week 5	LangGraph Agents & MCP Server
Tasks	• Build LangGraph agent state and agent graph • Implement Planner, Vision, Marketing, Support, Tool agents • Build MCP server tools, connect agents, save logs
Deliverables	✓ Agent workflows working ✓ MCP tools working ✓ Agent logs saved
Week 6	MLOps, Polish & Demo
Tasks	• Add MLflow tracking, prompt versioning, eval metrics • Improve frontend UI, add README documentation • Record demo video, prepare final presentation
Deliverables	✓ Final working MVP ✓ GitHub repository ✓ Demo video ✓ Final presentation

11. Evaluation Metrics

VLM Evaluation	JSON validity rate · Category accuracy · Attribute extraction accuracy · Visible text extraction quality · Description usefulness
RAG Evaluation	Retrieval accuracy · Recall@k · Similar product relevance · Grounded answer quality · Hallucination rate

Agent Evaluation	Successful workflow completion rate · Number of failed tool calls · Latency per workflow · Correct tool usage · Human review acceptance rate
Marketing Evaluation	SEO title quality · Ad copy relevance · Product information correctness · Seller approval rate

12. Risk Analysis

Risk	Mitigation
VLM slow on laptop	Use 3B model only; resize images before sending; allow manual editing of extracted data
VLM returns invalid JSON	Strict prompts; JSON parsing cleanup; Pydantic validation; fallback to editable draft
Limited laptop memory	Only PostgreSQL and Qdrant in Docker; run backend/frontend directly; avoid large models
Agent complexity	Deterministic LangGraph workflows; simple MCP tools; human review before saving outputs
Hallucinated answers	RAG context only; grounding checks; explicit 'insufficient data' response when uncertain

13. Cost Plan

The entire MVP is developed using a **zero-cost local setup**. All components run on the developer's machine using free and open-source tools:

- Qwen2.5-VL 3B via Ollama (free, local)
- PostgreSQL via Docker (free)
- Qdrant via Docker (free)
- FastAPI, Next.js, MLflow (all open-source)

Estimated development cost: \$0

14. Future Enhancements

- Larger VLM (Qwen2.5-VL 7B) with cloud GPU inference
- Shopify / WooCommerce integration for direct publishing
- WhatsApp Business API and email automation
- Real billing system and multi-tenant SaaS deployment
- Advanced analytics dashboard with seller insights
- Fine-tuning on e-commerce product data

- Better OCR, brand detection, and image quality scoring
- Automatic price comparison across platforms

15. Conclusion

The **Multimodal AI E-commerce Intelligence & Automation Agent** is a realistic, production-style AI SaaS project for e-commerce sellers. It combines vision-language models, RAG, agentic workflows, MCP tools, vector search, and MLOps concepts into a single coherent system.

The project is achievable in six weeks using a zero-cost local setup, satisfies the VLM requirement with Qwen2.5-VL 3B through Ollama, and can serve as an academic project, a GitHub portfolio showcase, and a foundation for a future SaaS startup.

Final Stack: Next.js · FastAPI · PostgreSQL · Qdrant · Ollama · Qwen2.5-VL 3B · LangGraph · MCP Server · MLflow